

Server & Data Center Administration

Comandos Básicos - BASH, CMD e PowerShell

Introdução às Interfaces de Linha de Comando

Windows 11: No Windows 11, temos duas interfaces principais de linha de comando:

- **CMD (*Command Prompt*):** O CMD, também conhecido como Prompt de Comando ou Command Line, é uma interface de linha de comando padrão do sistema operacional Windows. Ele permite que os usuários executem comandos para realizar tarefas administrativas, como: Navegar por pastas; Copiar/mover/deletar arquivos; Rodar programas e scripts; e Executar comandos do sistema
- **PowerShell:** O PowerShell é uma ferramenta mais poderosa e moderna que o CMD. Ele também é uma interface de linha de comando, mas com recursos adicionais, como: Suporte a scripts mais avançados; Uso de cmdlets (comandos especializados do PowerShell); Automação de tarefas do sistema operacional; e Integração com .NET e outros serviços do Windows
- **Para abrir:**
 - **CMD:** Win + R → digite cmd → Enter
 - **PowerShell:** Win + X → selecione "Windows PowerShell" ou "Terminal"

Introdução às Interfaces de Linha de Comando

Bash (acrônimo de *Bourne Again SHell*) é um interpretador de linha de comando usado principalmente em sistemas Linux, como o Ubuntu. Ele permite que o usuário interaja com o sistema operacional digitando comandos em texto. O Bash é o shell padrão na maioria das distribuições Linux, incluindo o Ubuntu.

- **Funções principais do Bash:**
 - Executar comandos do sistema (como copiar arquivos, listar diretórios, etc.)
 - Criar e rodar scripts automatizados (.sh)
 - Manipular variáveis, loops, condições e funções (como numa linguagem de programação)
 - Navegar entre diretórios e gerenciar arquivos
 - Controlar permissões e processos
- **Para abrir:** Ctrl + Alt + T

Introdução às Interfaces de Linha de Comando

Microsoft Azure

A interação com o Azure pode ser feita através de:

- **Azure CLI:** ferramenta de linha de comando multiplataforma
- **Azure PowerShell:** módulos PowerShell para Azure
- **Azure Cloud Shell:** shell baseado em navegador

Navegação Básica no Sistema de Arquivos

Operação	Windows (CMD)	Windows (PowerShell)	Ubuntu (Bash)
Mostrar diretório atual	cd	Get-Location ou pwd	pwd
Listar conteúdo	dir	Get-ChildItem ou ls	ls
Listar com detalhes	dir /a	Get-ChildItem - Force ou ls -la	ls -la
Mudar diretório	cd caminho	Set-Location caminho ou cd caminho	cd caminho
Voltar um nível	cd ..	cd ..	cd ..
Ir para raiz	cd \	cd \	cd /
Ir para home	cd %USERPROFILE%	cd ~	cd ~ ou cd
Limpar tela	cls	Clear-Host ou cls	clear

Manipulação de Arquivos

Operação	Windows (CMD)	Windows (PowerShell)	Ubuntu (Bash)
Criar arquivo vazio	type nul > arquivo.txt	New-Item -ItemType File arquivo.txt	touch arquivo.txt
Exibir conteúdo	type arquivo.txt	Get-Content arquivo.txt ou cat arquivo.txt	cat arquivo.txt
Visualizar com paginação	more arquivo.txt	Get-Content arquivo.txt more	less arquivo.txt
Copiar arquivo	copy origem.txt destino.txt	Copy-Item origem.txt destino.txt	cp origem.txt destino.txt
Mover/renomear	move origem.txt destino.txt	Move-Item origem.txt destino.txt	mv origem.txt destino.txt
Excluir arquivo	del arquivo.txt	Remove-Item arquivo.txt	rm arquivo.txt
Comparar arquivos	fc arquivo1.txt arquivo2.txt	Compare-Object (Get-Content file1.txt) (Get-Content file2.txt)	diff arquivo1.txt arquivo2.txt
Editar arquivo	notepad arquivo.txt	notepad arquivo.txt	nano arquivo.txt ou vim arquivo.txt

Manipulação de Diretórios

Operação	Windows (CMD)	Windows (PowerShell)	Ubuntu (Bash)
Criar diretório	mkdir pasta	New-Item -ItemType Directory pasta ou mkdir pasta	mkdir pasta
Criar árvore de diretórios	mkdir pasta1\pasta2\pasta3	mkdir -p pasta1/pasta2/pasta3	mkdir -p pasta1/pasta2/pasta3
Remover diretório vazio	rmdir pasta	Remove-Item pasta	rmdir pasta
Remover diretório com conteúdo	rmdir /s /q pasta	Remove-Item -Recurse -Force pasta	rm -rf pasta
Copiar diretório	xcopy /E /I origem destino	Copy-Item -Recurse origem destino	cp -r origem destino

Permissões e Atributos

No Windows (CMD / PowerShell): O comando *attrib* permite visualizar e modificar atributos de arquivos, como somente leitura, oculto, entre outros.

Exibe os atributos do arquivo, como:

R = Read-only
(somente leitura)

H = Hidden (oculto)

A = Archive
(arquivável)

S = System (arquivo do sistema)

```
1  # Verificar atributos
2  attrib arquivo.txt
3
4  # Definir atributo de somente leitura
5  attrib +r arquivo.txt
6
7  # Remover atributo de somente leitura
8  attrib -r arquivo.txt
9
10 # PowerShell: Ver permissões
11 Get-Acl arquivo.txt
12
13 # PowerShell: Definir permissões
14 $acl = Get-Acl arquivo.txt
15 $accessRule = New-Object System.Security.AccessControl.FileSystemAccessRule("Usuario","FullControl","Allow")
16 $acl.SetAccessRule($accessRule)
17 $acl | Set-Acl arquivo.txt
```

Permissões e Atributos



```
1  # Ver permissões
2  ls -l arquivo.txt
3
4  # Modificar permissões
5  chmod 644 arquivo.txt # rw-r--r--
6
7  # Alterar proprietário
8  chown usuario:grupo arquivo.txt
9
10 # Permissões recursivas
11 chmod -R 755 pasta/
```

Pesquisa e Filtragem

Operação	Windows (CMD)	Windows (PowerShell)	Ubuntu (Bash)
Buscar arquivos	<code>dir /s *.txt</code>	<code>Get-ChildItem -Recurse -Filter "*.txt"</code>	<code>find . -name "*.txt"</code>
Buscar texto em arquivos	<code>findstr "texto" *.txt</code>	<code>Select-String -Pattern "texto" -Path *.txt</code>	<code>grep "texto" *.txt</code>
Busca avançada	<code>where /r c:\ "*.txt"</code>	<code>Get-ChildItem -Path C:\ -Recurse -Filter "*.txt"</code>	<code>find / -type f -name "*.txt"</code>
Contagem de linhas	<code>find /c /v "" arquivo.txt</code>	<code>(Get-Content arquivo.txt).Length</code>	<code>wc -l arquivo.txt</code>

Gerenciamento de Processos

Operação	Windows (CMD)	Windows (PowerShell)	Ubuntu (Bash)
Listar processos	tasklist	Get-Process	ps aux
Finalizar processo	taskkill /PID 1234	Stop-Process -Id 1234	kill 1234
Forçar finalização	taskkill /F /PID 1234	Stop-Process -Id 1234 -Force	kill -9 1234
Executar em background	start programa.exe	Start-Process programa.exe	programa &

Permissionamento

O que é SSH?

SSH (*Secure Shell*) é um protocolo de rede que permite a comunicação segura entre dispositivos. Ele é usado para enviar comandos, controlar servidores remotos e transferir arquivos.

O SSH usa criptografia para autenticar e criptografar as conexões, garantindo que os dados não sejam perdidos ou interceptados. O SSH é muito utilizado em ambientes *Unix-like*, como Linux e MacOS. Para usuários do Windows, é possível usar clientes SSH como o *Putty*.

Para que serve o SSH? Controlar servidores remotamente, Gerenciar a infraestrutura, Transferir arquivos, Acessar e modificar sites, Desenvolver páginas da web.

Como funciona o SSH? O SSH usa pares de chaves públicas e privadas para criptografar a conexão entre os dispositivos. O usuário gera a chave privada e a mantém em segredo, enquanto a chave pública é colocada nos dispositivos que devem permitir o acesso.

2. Permissionamento de Arquivos e Pastas

Linux/Bash

Visualizando permissões: `ls -l`

Significado:

- Exemplo: `-rw-r--r--`
 - -: Arquivo regular
 - r: read (leitura)
 - w: write (escrita)
 - x: execute (execução)

2. Permissionamento de Arquivos e Pastas

No Windows, as permissões são gerenciadas via ACLs (*Listas de Controle de Acesso*).

attrib: Altera atributos básicos de arquivos.

- `attrib +R arquivo.txt` # Define como "somente leitura"
- `attrib -H documento.conf` # Remove atributo "oculto"

2. Permissionamento de Arquivos e Pastas

icacs: Gerencia permissões avançadas (usuários/grupos).

- `icacs pasta_projeto /grant Usuário:(F) # Concede controle total`
- `icacs arquivo.txt /deny Guest:(R) # Nega leitura ao convidado`

Considerar:

- **F:** Full control
- **R:** Read
- **W:** Write

2. Permissionamento de Arquivos e Pastas

No Linux, as permissões usam usuário, grupo e outros (leitura r, escrita w, execução x).

chmod: Altera permissões (modo numérico ou simbólico).

- `chmod 755 script.sh` # rwx para dono, r-x para outros (7=rwx, 5=r-x)
- `chmod u+x arquivo` # Adiciona permissão de execução ao dono

2. Permissionamento de Arquivos e Pastas

`chown`: Altera dono/grupo.

- `sudo chown usuario:grupo arquivo.txt`

Ver permissões:

- `ls -l` # Exibe permissões no formato "-rwxr--r--"

Busca

3. Busca e Localização de Arquivos

No Windows:

Encontrar arquivos:

- `dir /s *.txt` # Busca todos .txt no diretório atual e subpastas
- `dir /s /b "relatorio*"` # Lista caminhos completos de arquivos começando com "relatorio"

3. Busca e Localização de Arquivos

Buscar texto em arquivos:

- `findstr /s /i "palavra-chave" *.log # Busca case-insensitive em todos .log`

3. Busca e Localização de Arquivos

Localizar executáveis no PATH:

- `where python` # Mostra o caminho do executável "python"

3. Busca e Localização de Arquivos

No Linux:

Encontrar arquivos:

- `find /home -name "*.pdf"` # Busca PDFs em /home
- `find /var/log -size +10M` # Arquivos maiores que 10MB

3. Busca e Localização de Arquivos

Buscar texto em arquivos:

- `grep -r "erro" /var/log/` # Busca "erro" recursivamente
- `grep -i "warning" syslog` # Case-insensitive

3. Busca e Localização de Arquivos

Atualizar base de dados para locate:

- `sudo updatedb` # Atualiza banco de dados para busca rápida
- `locate relatorio` # Encontra arquivos chamados "relatorio"

Localização

4. Localização

No Windows:

Navegação básica:

- `cd Documents` # Entra na pasta "Documents"
- `cd ..` # Volta um nível
- `dir` # Lista arquivos e pastas

4. Localização

Caminho atual:

- `echo %cd%` # Exibe o diretório atual

4. Localização

No Linux:

Navegação básica:

- `cd ~/Documentos` # Vai para a pasta "Documentos" do usuário
- `pwd` # Exibe o caminho atual
- `ls` # Lista conteúdo

4. Localização

Encontrar caminho de comandos:

- `which python3` # Mostra o caminho do executável
- `type -a ls` # Exibe a localização do comando "ls"

Para Lembrar

Tarefa	Windows (CMD)	Bash (Linux)
Permissões	attrib, icacls	chmod, chown
Buscar arquivos	dir /s, findstr	find, locate
Buscar texto	findstr	grep
Caminho atual	echo %cd%	pwd

Conclusão

Permissões, busca e navegação é essencial para administrar sistemas e automatizar tarefas. Pratique os comandos e explore opções avançadas (ex: `find -exec`, `icacls /save`).

Para se aprofundar, estude:

- Scripting em Batch/PowerShell.
- Permissões especiais no Linux (ex: sticky bit, ACLs).

Dica de segurança: Evite conceder permissões amplas (ex: `chmod 777` ou `icacls /grant Todos:(F)`). Sempre restrinja ao mínimo necessário!

Sequenciamento de comandos
para execução de tarefas
complexas.

O que é Sequenciamento de Comandos?

É o processo de encadear múltiplos comandos no terminal, de modo que eles sejam executados em sequência lógica, permitindo a realização de tarefas mais complexas do que seria possível com comandos isolados.

Esse recurso é fundamental para:

- Automatizar rotinas administrativas
- Criar scripts reutilizáveis
- Otimizar tempo e reduzir erros manuais

Conceitos Básicos de Sequenciamento

a) Execução em sequência independente

- No Linux (**bash**): *comando1 ; comando2*
- No Windows (**CMD**): *comando1 & comando2*

Descrição: Executa todos os comandos, independentemente do sucesso ou falha do anterior.

- **; (Sequência):** Executa comandos em sequência, independente do resultado.
- **Exemplo:** *echo "Início"; ls; echo "Fim" #Exibe mensagens antes e depois de listar arquivos*

Conceitos Básicos de Sequenciamento

b) Execução condicional com sucesso

Linux (Bash) e Windows (CMD): *comando1 && comando2*

Descrição: O segundo comando só será executado se o primeiro for bem-sucedido (ou seja, retornar "sucesso").

- **&& (AND lógico):** Executa o próximo comando apenas se o anterior for bem-sucedido.
- **Exemplo Windows (CMD):** *mkdir pasta_nova && cd pasta_nova # Cria a pasta e entra nela se a criação for bem-sucedida*
- **Exemplo Linux (Bash):** *sudo apt update && sudo apt upgrade # Atualiza a lista de pacotes e depois os pacotes*

Conceitos Básicos de Sequenciamento

c) Execução condicional com falha

Linux (Bash) e Windows (CMD): *comando1 || comando2*

Descrição: O segundo comando só será executado se o primeiro falhar.

- **|| (OR lógico):** Executa o próximo comando apenas se o anterior falhar.
- **Exemplo Windows (CMD):** *ping google.com || echo "Falha na conexão"* # Exibe mensagem se o ping falhar
- **Exemplo Linux (Bash):** *systemctl start apache2 || journalctl -xe* # Tenta iniciar o Apache; se falhar, exibe logs

Conceitos Básicos de Sequenciamento

d) Pipelines (|)

Linux (Bash) e Windows (PowerShell): *comando1 | comando2*

Descrição: Transfere a saída do primeiro comando como entrada para o segundo. É uma forma de conectar comandos de forma funcional, formando "cadeias de processamento".

- | (**Pipes Lógico**): Redireciona a saída de um comando para a entrada de outro.
- Exemplo Windows (PowerShell): *Get-ChildItem | where-Object { \$_.Length -gt 1MB }* # Lista arquivos maiores que 1MB
- Exemplo Linux (Bash): *ps aux | grep "nginx"* # Lista processos e filtra por "nginx"

Conceitos Básicos de Sequenciamento

e) Redirecionamento de saída

- > (sobrescreve)
- >> (anexa)

Descrição: Permite direcionar a saída de comandos para arquivos de log, relatórios ou scripts.

- **Exemplo Linux (Bash):** Criar backup e registrar em log
- *tar -czf backup.tar.gz ~/Documentos && echo "Backup realizado em \$(date)" >> backup.log*
- **Explicação :** Compacta arquivos e registra data/hora no log se o backup funcionar.

Técnicas Avançadas

1. Substituição de Comandos

Usa a saída de um comando como argumento para outro.

- Exemplo Linux (Bash):
 - *echo "Hoje é \$(date)"* # Insere a data atual na string
 - *tar -czvf backup-\$(date +%Y%m%d).tar.gz /dados* # Cria backup com data no nome
- Exemplo Windows (PowerShell):
 - *Write-Output "Arquivos: \$(Get-ChildItem | Measure-Object | Select-Object -ExpandProperty Count)"*

Técnicas Avançadas

2. Variáveis Temporárias

Armazene resultados intermediários para reutilização.

- Exemplo Linux (Bash):
 - `total=$(du -sh /home | awk '{print $1}')` # Armazena o tamanho do diretório /home
 - `echo "Tamanho total: $total"`
- Exemplo Windows (PowerShell):
 - `$processos = Get-Process | where-Object { $_.CPU -gt 10 }`
 - `$processos | Export-Csv -Path "processos_pesados.csv"`

Exemplos Aplicáveis

Exemplo: Monitorar e Finalizar Processos

- **Exemplo Linux (Bash):** Mata processos do Chrome que consomem mais de 20% de CPU
 - `ps aux | awk '/chrome/ && $3 > 20 { print $2 }' | xargs kill -9`
- **Exemplo Windows (PowerShell):** Encerra processos do Chrome com uso alto de CPU
 - `Get-Process chrome | where-Object { $_.CPU -gt 20 } | Stop-Process -Force`

Exemplos Aplicáveis

Exemplo: Análise de Logs

- **Exemplo Linux (Bash):** Conta quantas vezes "ERROR" apareceu nos logs de hoje
 - `grep "ERROR" /var/log/syslog | grep "$(date +%Y-%m-%d)" | wc -l`
- **Exemplo Windows (CMD):** Conta linhas com "ERROR" em todos os .log
 - `findstr /c:"ERROR" C:\logs\*.log | find /c ":"`

Exemplos Aplicáveis

Exemplo: Backup Automatizado

- **Exemplo Linux (Bash):** Compacta arquivos modificados nas últimas 24h e envia para um servidor
 - `find /dados -type f -mtime -1 | tar -czvf backup-$(date +%F).tar.gz -T - && scp backup-*.tar.gz user@servidor:/backup`
- **Exemplo Windows (PowerShell):** Compacta logs antigos e os exclui
 - `Get-ChildItem -Path "C:\logs" -Filter *.log | where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-7) } | Compress-Archive -DestinationPath "backup-$(Get-Date -Format 'yyyyMMdd').zip" -PassThru | Remove-Item`

Técnicas de Depuração

Depuração (*Debugging*) é o processo de identificar, analisar e corrigir erros (*bugs*) em um programa de computador, *script* ou sistema, visando garantir que ele funcione conforme o esperado. Esses erros podem causar comportamentos inesperados, falhas de execução, ou resultados incorretos, e a depuração é essencial para melhorar a qualidade, eficiência e segurança do software.

Redirecionamento de Saída

- **Exemplo Linux (Bash):** Saída padrão em arquivos.txt, erros em erros.txt
 - `ls > arquivos.txt 2> erros.txt`
- **Exemplo Windows (CMD):** Redireciona saída e erros para o mesmo arquivo
 - `dir > arquivos.txt 2>&1`

Importância Prática

Ao utilizar o sequenciamento de comandos, será possível:

1. Automatizar fluxos como **backups**, **atualizações** e **monitoramentos**.
2. Evitar **retrabalho** com rotinas manuais **repetitivas**.
3. Melhorar o **desempenho** em tarefas de administração de sistemas.
4. Criar **scripts robustos**, que tratam **erros** e registram **logs**.
5. Aplicações Típicas
 1. **DevOps/SysAdmin**: Automatização de **deploy**, **logs**, **backups**.
 2. **Desenvolvimento**: Execução de **testes**, **builds** e **limpeza** de arquivos.
 3. **Análise de dados**: **Filtragem**, **ordenação** e **transformação** de dados **textuais**.

Conclusão

O sequenciamento de comandos representa um **poderoso paradigma de automação** e eficiência computacional.

Ao conectar comandos simples com operadores lógicos e redirecionadores, é possível construir **rotinas complexas e inteligentes** dentro do próprio terminal, sem necessidade de linguagens de programação completas.